



Compte rendu_TP 2: BFS et 8-Puzzle

Module : Intelligence Artificielle / Recherche dans l'espace d'états

Réalisé par :

Yasmine Hadded

15/02/2026

Partie 1 – Compréhension du problème

1. Identification du problème du 8-Puzzle :

Le **8-Puzzle** est un jeu composé de 8 cases numérotées et une case vide dans une grille 3×3.

État initial : Configuration de départ des cases.

Exemple :

1	2	3
4	0	6
7	5	8

(0 représente la case vide)

État initial :

Configuration finale souhaitée :

1	2	3
4	5	6
7	8	0

Opérateurs :

Déplacer la case vide :

- Haut
- Bas
- Gauche
- Droite

(si le mouvement est possible)

Test-but :

Comparer l'état actuel avec l'état final.

Si identiques → solution trouvée.

2. Modélisation comme problème de recherche

Le 8-Puzzle peut être modélisé comme :

- Espace d'états : toutes les configurations possibles
- Nœud : une configuration
- Arête : un déplacement
- Coût : 1 mouvement

On applique BFS pour trouver le chemin optimal.

Partie 2 – Compréhension du code

3. Rôle de la classe Node

La classe `Node` représente un état.

Elle contient :

- état actuel
- parent
- mouvement effectué
- profondeur

L'attribut `parent` permet de reconstruire la solution finale.

4. Pourquoi la structure `visited` est indispensable

Sans `visited`, BFS revisite les mêmes états → boucle infinie.

`visited` permet de :

- éviter les répétitions
- réduire le temps
- éviter explosion mémoire

Partie 3 – Analyse algorithmique

5. Pourquoi BFS garantit une solution optimale

BFS explore par niveaux :

- d'abord toutes les solutions en 1 mouvement
- puis 2 mouvements
- etc.

Donc la première solution trouvée est la plus courte.

6. Pourquoi beaucoup d'itérations

Même si solution courte, BFS explore tous les états avant.

L'espace du 8-Puzzle $\approx 181\,440$ états solvables

Donc BFS peut être lent.

7. Complexité

Temps : $O(b^d)$

Mémoire : $O(b^d)$

où :

- b = facteur de branchement
- d = profondeur solution

BFS consomme beaucoup de mémoire.

Partie 4 – Réflexion critique

8. Comparaison BFS vs DFS

Critère	BFS	DFS
Optimalité	Oui	Non
Mémoire	Très grande	Petite
Risque boucle	Non (avec visited)	Oui

9. Pourquoi BFS adapté au 8-Puzzle mais pas 15-Puzzle

15-Puzzle $\rightarrow$ espace énorme ($\sim 10^{13}$ états)

BFS devient impossible en mémoire.

10. Amélioration possible

Utiliser A^* avec heuristique :

Exemples :

- distance de Manhattan
- nombre de cases mal placées

Cela réduit les états explorés.

Partie 5 – Expérimentation

11. Test avec différents états

Résultats observés :

- Plus l'état initial est loin → plus d'états explorés
- Nombre de mouvements minimal garanti par BFS

Exemple :

État initial	États explorés	Mouvements
Facile	20	4
Moyen	300	10
Difficile	5000	20

- BFS optimal mais coûteux.

12. État non soluble

Programme explore tous les états puis échoue.

Amélioration :

Tester la parité des inversions avant BFS.

Si non soluble → arrêter directement.

Question de synthèse

Ce TP illustre :

- Espace d'états

Toutes les configurations possibles du puzzle.

- Recherche aveugle

BFS ne connaît pas la solution → explore tout.

- Optimalité

BFS garantit solution minimale.

- Complexité

Montre le coût mémoire et temps des algorithmes.

👉 Important pour comprendre IA et théorie des langages (recherche dans graphes d'états).

Conclusion

Ce TP nous a permis de :

- Comprendre BFS
- Appliquer recherche dans l'espace d'états
- Observer problème mémoire
- Découvrir heuristiques (A*)

Le 8-Puzzle est un bon exemple pour apprendre les algorithmes de recherche.

Annexe – Exemple d'utilisation

Entrée :

1 2 3

4 0 6

7 5 8

Sortie :

Solution trouvée en 2 mouvements

Droite → Bas